SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	K NAVYA	**Reg. No.:**	10/10/2026
Date:	192371022	**Duration:**	60 minutes

Code of Conduct

I K NAVYA(192371022)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification	Selects the most suitable SDLC/model/fram	Suitable methodology selected with	Methodology selected but justification is	Outcomes, limitations, or future enhancements

(10%)	ework and provides strong technical justification.	adequate justification.	weak.	are missing; report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

SMART SCHOLARSHIP AND FINANCIAL AID MANAGEMENT PLATFORM

1. Understanding the Problem Statement

1.1 Problem in Our Own Words

Educational institutions offer scholarships and financial assistance through government schemes, institutional funds, trusts, charities, and private sponsors. Each opportunity may have different rules related to academic performance, family income, course, year of study, residence, category, or special circumstances. Students must discover these opportunities, understand the conditions, prepare supporting documents, and submit applications before deadlines. Administrators then need to check eligibility, verify documents, obtain approvals, and arrange payments.

When these activities are handled through notices, paper forms, spreadsheets, email, or separate portals, information can be scattered and difficult to track. Students may not know which schemes match their circumstances, while staff may repeatedly check the same information and struggle to identify incomplete applications. The proposed platform brings these activities into one secure system. It compares student profiles with configured scholarship rules, recommends potentially suitable opportunities, supports online applications, and records each review and payment stage. Automated recommendations assist staff; final decisions remain with authorized reviewers.

1.2 Objectives of the Proposed System

- Maintain a centralized, searchable catalogue of scholarships, grants, fee concessions, and other aid.
- Allow authorized staff to configure eligibility criteria, required documents, deadlines, award values, and approval rules.
- Match student profile information against criteria and notify students of potentially suitable opportunities.
- Enable online application submission, document upload, correction requests, and application-status tracking.
- Support document verification, reviewer comments, approval or rejection, and a traceable decision history.
- Record approved award amounts, payment schedules, disbursement dates, and payment status.
- Send timely reminders about deadlines, missing documents, decisions, and disbursements.
- Generate accurate reports for management, sponsors, audits, and institutional planning.

1.3 Scope of the Problem

The initial scope includes student registration and profiles, scholarship publishing, rule-based eligibility screening, application submission, document management, review and approval workflows, notifications, disbursement tracking, and reporting. Access will depend on user roles. The platform records decisions and payment information, but it does not itself guarantee funding, replace institutional policy, or automatically transfer money unless an approved payment integration is added. Complex exceptions and final awards remain subject to human review. Integration with student information systems, identity services, or banking systems may be introduced where permitted.

2. Key Stakeholders and Challenges

Stakeholders are the people and organizations that use, manage, fund, or are affected by the platform. Clear responsibilities and access permissions are essential because applications contain personal, academic, and financial information.

Stakeholder	Roles and Responsibilities	Major Challenges
Students / Applicants	Create profiles; discover schemes; submit applications and documents; respond to correction requests; track decisions and payments.	Limited awareness, confusing criteria, document collection, internet/device access, language barriers, and missed deadlines.
Scholarship / Financial Aid Office	Publish schemes; configure rules; check completeness; coordinate reviews; maintain records and answer queries.	High application volume, repetitive manual checks, inconsistent records, and peak-period workload.
Department Faculty / Academic Staff	Confirm enrollment, course, semester, attendance or academic performance where required; provide recommendations.	Delayed confirmations, different record formats, and competing responsibilities.
Document Verifiers	Check authenticity, legibility, validity, and consistency of income, identity, academic, or other evidence.	Fraudulent or expired documents, unclear scans, privacy risks, and time-consuming verification.
Selection Committee / Approving Authority	Review eligible applications, apply policy, record decisions and reasons, and resolve exceptions.	Need for fair, explainable decisions, conflicts of interest, and incomplete evidence.
Finance / Accounts Team	Confirm approved amounts, schedule payments, reconcile transactions, and record failed or completed disbursements.	Incorrect bank details, payment delays, reconciliation errors, and changes to award amounts.
Institution Management	Set policy, assign budgets, monitor outcomes, approve access, and review reports.	Limited visibility across schemes, audit readiness, and measuring impact.
Sponsors / Government Agencies	Provide funding and scheme rules; receive permitted reports; monitor use of funds.	Different reporting formats, compliance obligations, and delayed status information.
IT Administrators / Support	Manage accounts, access controls, backups, integrations, availability, and incident response.	Cybersecurity threats, system downtime, data migration, and support requests.

2.1 Cross-Stakeholder Challenges

- Information gaps: scholarship details may be spread across notice boards, websites, emails, and office records.
- Fairness and accuracy: unclear rules or inconsistent interpretation can produce incorrect eligibility decisions.
- Privacy and trust: financial and identity documents must only be visible to authorized personnel.
- Coordination: application review, academic confirmation, committee decisions, and payment processing depend on multiple teams.
- Accessibility: the service should work on mobile devices, support clear language, and accommodate users with disabilities.

3. Analysis of the Current Approach

3.1 Existing Process

In many institutions, scholarship administration is partly manual or distributed across several systems. A scheme is announced through a notice, email, website, or classroom communication. Students read the announcement and decide whether they qualify. They collect certificates, complete a paper or online form, and submit it to an office. Staff check the application and supporting documents, request corrections, and forward the file to a department or committee. After a decision is made, the result is communicated to the student. The finance team then processes the award and maintains a separate payment record. Reports are compiled from spreadsheets, files, and office registers.

3.2 Strengths of the Current Approach

- It can be inexpensive to start, especially for a small number of applications.
- Staff can apply institutional knowledge and use judgment for unusual cases.
- Paper signatures and familiar office procedures may already satisfy some local approval requirements.
- Existing spreadsheets and forms may be easy to adapt for a small scheme.

3.3 Limitations and Gaps

- Fragmented information: students cannot easily compare all active opportunities in one place.
- Manual eligibility screening: staff must repeatedly compare profiles with multiple rules, increasing workload and the risk of human error.
- Weak deadline visibility: notices can be missed, and reminders are not consistently personalized.
- Repeated data entry: the same student, course, and financial details may be entered into multiple forms or files.
- Limited status transparency: applicants may need to visit or contact an office to learn whether their file is pending, incomplete, approved, or rejected.
- Document handling risks: paper copies and email attachments can be misplaced, duplicated, or shared with unintended recipients.
- Slow coordination: applications can remain idle while waiting for verification, committee review, or finance processing.
- Difficult reporting: totals, rejection reasons, demographic reach, and payment status may require manual consolidation.
- Weak audit trail: it may be difficult to establish who changed a record, verified a document, or approved an award and when.

3.4 Problems to Address

The new system should create one reliable record for each scholarship and application, apply published eligibility rules consistently, make application status visible, and preserve accountability at each step. It must distinguish between a preliminary match and a final eligibility decision. It should also prevent unauthorized access, retain decision reasons, and provide a clear correction and appeal route where policy allows. Automation should reduce routine effort without removing human judgment from sensitive or exceptional cases.

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview

The proposed platform is a secure web-based application, responsive on mobile and desktop, with role-based dashboards for students, scholarship staff, verifiers, approvers, finance teams, sponsors, and administrators. A centralized database stores scheme details, student profiles, applications, document metadata, decisions, notifications, and disbursement records. A rules engine compares profile attributes with eligibility conditions and displays a match explanation, such as "potential match—minimum academic score met; income proof pending." Staff can review the evidence and override or correct a preliminary result with a recorded reason.

4.2 Major Functional Modules

- **User and Role Management:** Registration, authentication, profile maintenance, role permissions, and account recovery.
- **Scholarship Catalogue:** Create, edit, publish, close, and archive schemes; manage amounts, quotas, criteria, deadlines, and required evidence.
- **Eligibility Rules Engine:** Evaluate configured rules against student profiles; explain matched and unmet criteria; flag missing information for human review.
- **Application Management:** Save drafts, submit applications, validate required fields, issue reference numbers, and support resubmission when permitted.
- **Document Management:** Upload files, check file type and size, record versions, assign verification status, and request replacements securely.
- **Verification and Approval Workflow:** Route applications to the correct verifier and approver; record comments, decisions, timestamps, and reasons; escalate overdue tasks.
- **Notifications:** Send email, SMS, or in-platform reminders for deadlines, missing documents, status changes, and payment updates, subject to configured channels.
- **Disbursement Tracking:** Record approved amounts, installments, payment references, failed transactions, reconciliation, and completion status.
- **Reports and Audit:** Produce application, eligibility, approval, rejection-reason, deadline, fund utilization, and disbursement reports; preserve an audit log.

4.3 Typical Workflow

1) Staff publish a scheme and its criteria. 2) The platform checks student profiles and alerts potential matches. 3) The student reviews the details, uploads evidence, and submits an application. 4) The system validates required fields and sends the file to a verifier. 5) The verifier confirms documents or requests corrections. 6) The authorized committee or officer approves or rejects the application and records the reason. 7) Finance records the payment schedule and updates each disbursement. 8) The student receives status updates, and authorized users can generate reports.

4.4 Requirements

Functional Requirements

- The system shall create and maintain scholarship schemes, eligibility rules, deadlines, award values, and quotas.
- The system shall match student profile data to scheme criteria and identify potential opportunities.
- The system shall accept applications and supporting documents, validate required fields, and track versions.
- The system shall support verification, correction requests, approval/rejection, comments, and decision history.
- The system shall notify users of deadlines, missing items, workflow changes, and disbursement updates.
- The system shall record award amounts and payment status and generate role-appropriate reports.

Non-Functional Requirements

- Security and privacy: encrypted communication, secure file storage, least-privilege access, multifactor authentication for sensitive roles, and defined retention rules.
- Usability and accessibility: clear language, responsive screens, accessible forms, helpful validation, and understandable status labels.
- Reliability and performance: regular backups, recovery procedures, monitoring, and acceptable response times during application peaks.
- Scalability: support additional students, schemes, institutions, and concurrent applications without major redesign.

5. Methodology, Expected Outcomes, And Limitations

5.1 Recommended SDLC: Agile with Scrum

Agile development using Scrum is recommended because scholarship policies, user needs, integrations, and reporting requirements may evolve as students and administrative staff test the system. Scrum divides work into short sprints, typically one to three weeks, and delivers usable increments for feedback rather than waiting until the entire system is complete. A product backlog captures requirements, while sprint planning, daily coordination, reviews, and retrospectives help the team prioritize value and improve its process.

A practical sequence is: (1) discovery and stakeholder interviews; (2) prototype the catalogue and student journey; (3) build authentication, scheme setup, and profiles; (4) add eligibility matching and applications; (5) implement document verification and approvals; (6) add notifications, disbursement tracking, and reports; and (7) complete security testing, user acceptance testing, training, deployment, and support. Each increment should include testing and documentation. A pilot with a small set of schemes can expose confusing rules before institution-wide rollout.

5.2 Why Agile/Scrum Is Suitable

- Frequent stakeholder feedback helps ensure that screens, eligibility rules, and workflows match real institutional practice.
- Incremental delivery makes core features available earlier and reduces the risk of discovering major usability issues at the end.
- Prioritization allows urgent needs, such as deadline reminders or audit requirements, to be addressed first.
- Continuous testing and review improve quality, while retrospectives help the team respond to process problems.
- A maintained backlog and modular design support future changes to policies, integrations, and reporting.

5.3 Software Engineering Principles in Practice

Modularity separates catalogue, eligibility, applications, documents, workflow, notifications, finance, and reporting. Scalability is supported by stateless application services, database indexing, and the option to separate high-load components later. Maintainability comes from clear interfaces, reusable validation, configuration-driven rules, automated tests, and technical documentation. Reliability is strengthened by backups, monitoring, error handling, recovery plans, and reconciliation checks. Usability is addressed through accessible design, mobile-friendly pages, plain-language explanations, and visible application progress. Security is built in through role-based access, encryption, secure session handling, audit logs, periodic access reviews, and privacy-conscious data retention.

5.4 Expected Outcomes and Benefits

- Improved access: students can discover opportunities matched to their profiles in one place.
- Reduced administrative workload: rule-based screening, validation, reminders, and report generation reduce repetitive tasks.
- Faster processing: clear queues and workflow ownership help staff identify pending applications and overdue actions.
- Greater transparency: applicants can view current status, requests for correction, decisions, and payment updates.
- Fewer missed deadlines: targeted reminders provide timely prompts while keeping students responsible for submission.
- More accurate records: centralized data and audit logs support reporting, reconciliation, and accountability.

5.5 Risks, Limitations, and Mitigation

Incorrect or outdated student data can produce false matches; mitigate this with validation, data ownership, and periodic updates. Poorly configured rules can unfairly exclude students; test rules with representative cases and require authorized approval before publication. Sensitive documents may be exposed through weak access controls; apply encryption, least privilege, secure backups, and incident procedures. Users may resist a new process or have limited connectivity; provide training, mobile-friendly pages, helpdesk support, and assisted application options. Integrations and migration can add cost and delay, so begin with a pilot and agree on data formats early. Notifications may fail or be ignored, so show reminders inside the platform and make delivery status visible where supported.

The platform cannot guarantee that every eligible student will apply, prove document authenticity without suitable checks, or replace human judgment and institutional policy. Eligibility matching depends on the accuracy of configured criteria and available profile data. Automated decisions should therefore be explainable, reviewable, and subject to authorized human approval. The system also requires ongoing hosting, security updates, policy maintenance, and user support.

5.6 Future Enhancements and Conclusion

Future versions may add multilingual interfaces, mobile applications, integration with student information and verified identity services, sponsor portals, digital signatures, analytics to identify underserved groups, and carefully governed AI assistance for document classification or question answering. Any AI feature should be tested for bias, disclose its limitations, and never make unreviewed high-impact award decisions. Overall, the Smart Scholarship and Financial Aid Management Platform offers a structured, transparent, and secure way to manage the scholarship lifecycle—from discovery to disbursement—while keeping final decisions with authorized institutional personnel.